

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерного проектирования
Кафедра инженерной психологии и эргономики
Дисциплина: Компьютерные системы и сети (КСиС)

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему:

**ПРИЛОЖЕНИЕ, РЕАЛИЗУЮЩЕЕ СЕТЕВУЮ
КОМПЬЮТЕРНУЮ ИГРУ «ШАХМАТЫ»**

БГУИР КП 6-05-06 12 01 022 ПЗ

Студент

Джалилова Н.Р.

Руководитель

Болтак С.В.

Минск 2026

СОДЕРЖАНИЕ

Введение.....	5
1 Анализ предметной области	6
1.1 Обзор аналогов.....	6
1.2 Постановка задачи	8
2 Проектирование программного средства	9
2.1 Архитектура приложения.....	9
2.2 Проектирование сетевого взаимодействия	9
2.3 Моделирование игрового процесса.....	10
2.4 Проектирование пользовательского интерфейса	10
2.4.1 Главное меню	11
2.4.2 Игровой экран.....	11
2.4.3 Система управления ходами	11
3 Разработка программного средства	12
3.1 Реализация сетевой части	12
3.2 Реализация игровой логики.....	14
3.3 Реализация пользовательского интерфейса	16
4 Тестирование программного средства.....	18
4.1 Юнит-тестирование игровой логики	18
4.2 Интеграционное тестирование	19
5 Руководство пользователя	21
5.1 Описание пользовательского интерфейса.....	21
5.2 Управление программным средством.....	24
Заключение	26
Список использованных источников	27
ПРИЛОЖЕНИЕ А. Исходный код программы	28

ВВЕДЕНИЕ

Переход настольных игр в цифровую среду стал закономерным этапом развития индустрии развлечений, открывшим новые возможности для взаимодействия между игроками. Шахматы, будучи одной из самых древних и глубоких интеллектуальных игр, требуют особого подхода при реализации в сетевом формате, где точность передачи данных и синхронизация действий участников играют ключевую роль. Актуальность данного проекта обусловлена растущей популярностью сетевых игровых платформ, позволяющих объединять соперников независимо от их местоположения, а также необходимостью создания удобного инструмента для игры в шахматы с поддержкой сетевого взаимодействия.

В качестве цели курсовой работы выступает проектирование и реализация сетевой версии шахмат с графическим интерфейсом. Основной акцент делается на создании полноценного игрового продукта, который не только воспроизводит классические шахматные правила, но и оснащается современными сетевыми механизмами, обеспечивающими удобство проведения партий между удалёнными пользователями.

Для реализации поставленной цели предусмотрено выполнение следующих задач:

- 1 Реализовать сетевое взаимодействие между игроками с использованием двух протоколов: UDP для поиска доступных соперников в локальной сети и TCP для надёжной передачи ходов и синхронизации состояния игры.

- 2 Разработать полную игровую логику, включая поддержку всех правил шахмат (ходы фигур, взятие, рокировку, превращение пешки, контроль шаха и мата).

- 3 Создать интуитивно понятный пользовательский интерфейс, обеспечивающий наглядное отображение шахматной доски и фигур, а также комфортное управление процессом партии.

- 4 Реализовать систему блокировки интерфейса во время хода противника для предотвращения некорректных действий и соблюдения очерёдности ходов.

Итогом выполнения курсовой работы станет сетевое приложение, позволяющее двум игрокам, находящимся на разных устройствах, проводить шахматные поединки с соблюдением всех официальных правил и удобным визуальным представлением игрового процесса.

В разделе «Приложение А» будет представлен листинг кода с комментариями.

В графической части на листе 1 будет представлена схема алгоритма работы системы.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор аналогов

Современный рынок цифровых реализаций шахмат представлен большим количеством приложений, каждое из которых предлагает различные подходы к игровому процессу, интерфейсу и взаимодействию между пользователями.

В первую очередь следует рассмотреть приложение – «Chess – Play and Learn» [1], представляющее собой популярную платформу для игры и обучения шахматам. Внешний вид данного приложения представлен на рисунке 1.

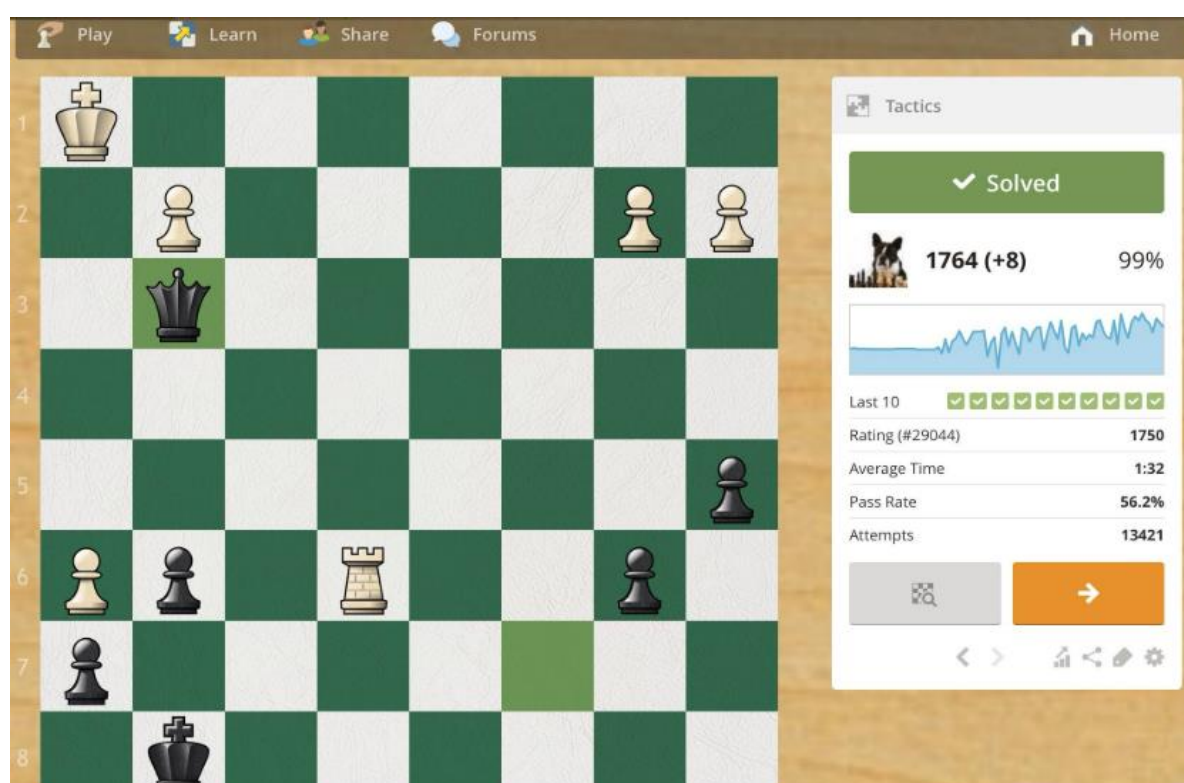


Рисунок 1 – Внешний интерфейс приложения «Chess – Play and Learn»

В приложении сохранены классические правила шахмат, однако вокруг них реализовано большое количество дополнительных возможностей, позволяющих охватить разные категории пользователей, но одновременно усложняющих восприятие основного игрового процесса.

Ключевыми характеристиками данного решения являются:

- использование обязательной авторизации;
- наличие рейтинговой системы и соревновательных режимов;
- встроенные обучающие материалы и аналитические инструменты;
- платную подписку, расширяющую функциональные возможности.

Таким образом, приложение ориентировано на развитие комплексного пользовательского опыта, объединяющего игру и обучение, однако обладает достаточно широким набором функций, выходящих за рамки классической реализации шахмат.

В качестве второго аналога следует рассмотреть приложение «Real Chess» [2]. Интерфейс данного приложения представлен на рисунке 2.



Рисунок 2 – Внешний интерфейс приложения «Real Chess»

Основные функциональные возможности приложения:

- онлайн-матчи с реальными игроками по всему миру;
- режим игры против ИИ с различными уровнями сложности;
- встроенная система подбора соперников;
- чат с противником во время партии;
- 2D и 3D варианты доски;
- внутриигровые покупки для расширения функционала.

Данная реализация представляет собой полноценную онлайн-шахматную игру с поддержкой 3D-графики, системой подбора соперников и дополнительными социальными функциями. Однако наличие различных режимов отображения и элементов взаимодействия между игроками делает приложение более сложным по структуре по сравнению с классическими шахматными реализациями и разрабатываемым проектом.

Таким образом, оба рассмотренных аналога содержат элементы монетизации, включая рекламу, внутриигровые покупки, подписки, ограничение функционала для free-to-play пользователей. Разрабатываемый проект не предусматривает платных функций и ориентирован на простую

сетевую реализацию шахмат с минималистичным интерфейсом, что обеспечивает более стабильный и «чистый» игровой процесс.

1.2 Постановка задачи

По результатам анализа существующих аналогов определен перечень требований к разрабатываемой сетевой реализации игры «Шахматы».

Функциональные требования:

1 Реализация правил классических шахмат:

- поддержка шахматной доски 8×8;
- полный набор шахматных фигур;
- корректная реализация ходов всех фигур;
- проверка допустимости ходов;
- обработка игровых состояний.

2 Организация сетевого взаимодействия [3]:

- поиск игровых сессий в локальной сети через UDP-бroadcast;
- обмен ходами между игроками посредством TCP-соединения.

3 Визуализация игрового процесса:

- отображение шахматной доски и фигур с использованием JavaFX;
- динамическое обновление состояния игры после каждого хода.

4 Реализация игрового процесса:

- блокировка интерфейса при ожидании хода соперника;
- завершение партии с формированием результата.

Нефункциональные требования:

1 Переносимость:

- поддержка операционных систем Windows, Linux и macOS;
- использование Java версии 21 и выше.

2 Надёжность:

- проверка входящих сетевых данных;
- обработка разрывов соединения;
- предотвращение некорректных пользовательских действий.

3 Эффективность:

- минимизация задержек при передаче ходов;
- оптимизация отрисовки интерфейса JavaFX;
- рациональное использование сетевых ресурсов.

4 Общие требования:

- минималистичный пользовательский интерфейс;
- возможность перезапуска игровой сессии без перезапуска приложения.

Разрабатываемая система обеспечивает стабильную сетевую игру для двух пользователей с реализацией классических правил шахмат и синхронизацией состояния партии в реальном времени. Для реализации используется Java 21+ и среда разработки IntelliJ IDEA.

2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

2.1 Архитектура приложения

Архитектура разрабатываемого приложения разделена на три основных модуля: серверный модуль, клиентский модуль и модуль передачи данных.

1. Серверный модуль (Server Core):

Отвечает за обработку игровой логики и взаимодействие между клиентами. Включает следующие подмодули:

- Модуль обработки запросов (Request Handler) – приём и разбор сообщений от клиентов;
- Модуль валидации (Validation Module) – проверка корректности действий игроков;
- Модуль управления игрой (Game Manager) – хранение и изменение состояния партии;
- Модуль синхронизации (Synchronization Module) – обновление состояния игры;
- Модуль рассылки (Broadcast Module) – отправка данных всем подключённым клиентам.

2. Клиентский модуль (Client Core):

Обеспечивает взаимодействие пользователя с игрой и связь с сервером. Содержит:

- Модуль пользовательского ввода (Input Handler) – обработка действий пользователя;
- Сетевой модуль (Network Client) – подключение к серверу и отправка запросов;
- Модуль обработки ответов (Response Handler) – приём и обработка сообщений от сервера;
- Модуль визуализации (Rendering Module) – отображение игрового процесса.

3. Модуль передачи данных (Data Transfer Objects) [4]:

Используется сервером и клиентом для унификации структуры обмена данными. Включает:

- Модели данных (Data Models) – представление доски, фигур и состояния игры;
- Сетевые сообщения (Message Protocol) – структуры сообщений для передачи по сети;
- Вспомогательные классы (Utilities) – общие структуры и утилиты.

2.2 Проектирование сетевого взаимодействия

Сетевое взаимодействие и организация обмена данными между участниками являются ключевым аспектом разработки игры «Шахматы».

В рамках проекта реализована сетевая модель, в которой одновременно используются протоколы UDP и TCP, каждый из которых выполняет отдельную задачу.

1 UDP-протокол применяется для автоматического поиска игровых серверов в локальной сети.

2 TCP-протокол отвечает за передачу основной игровой информации между клиентом и сервером. Через него осуществляется обмен ходами игроков, обновление состояния шахматной доски и синхронизация игровой логики.

Разделение функций между протоколами позволяет повысить эффективность работы системы: UDP обеспечивает быстрый процесс обнаружения подключений, а TCP гарантирует точную и надёжную доставку данных в правильной последовательности.

В процессе разработки также были предусмотрены сценарии обработки ошибок и нестандартных ситуаций, таких как:

- обрыв соединения между участниками игры;
- восстановление подключения к текущей игровой сессии;
- временная недоступность сервера;
- возникновение ошибок передачи данных.

Реализованные механизмы направлены на поддержание стабильности работы приложения даже при нестабильных сетевых условиях. Предложенная архитектура удовлетворяет требованиям к:

- устойчивости к сбоям и ошибкам сети;
- эффективному использованию пропускной способности;
- кроссплатформенности, обеспечиваемой использованием Java.

2.3 Моделирование игрового процесса

Разрабатываемая модель игрового процесса основывается на классических правилах шахмат и учитывает особенности сетевой реализации приложения, требования к отзывчивости пользовательского интерфейса, а также необходимость корректной обработки исключительных ситуаций.

В процессе проектирования особое внимание уделяется строгому соблюдению правил перемещения фигур, а также реализации надежного механизма проверки допустимости ходов с учетом игровых состояний. Дополнительно обеспечивается корректная синхронизация состояния игры между клиентами в сетевом режиме, что позволяет поддерживать целостность игрового процесса.

2.4 Проектирование пользовательского интерфейса

В рамках разработки сетевого приложения, реализующего компьютерную игру «Шахматы», планируется создание интуитивно

понятного графического пользовательского интерфейса с использованием фреймворка JavaFX [5]. Пользовательский интерфейс будет включать такие компоненты как: главное меню, игровой экран, систему управления ходами.

2.4.1 Главное меню

Главное меню представляет собой начальный экран приложения, предназначенный для выбора режима работы и подготовки к началу игровой сессии.

В интерфейсе предусмотрены основные элементы управления, включающие:

- «Создать комнату», обеспечивающую запуск новой сетевой партии;
- «Присоединиться к комнате», позволяющую присоединиться к уже существующей игровой сессии.

Также реализовано поле ввода имени игрока, которое используется для идентификации участников в сетевом режиме. Вводимые данные проходят проверку на корректность перед использованием.

Графическое оформление главного меню предусматривает использование фонового изображения и минималистичного дизайна элементов управления, что обеспечивает удобство восприятия интерфейса.

2.4.2 Игровой экран

Основная игровая зона будет содержать:

Центральную область с шахматной доской (8×8), с отображением всех фигур и состояния партии;

Информационные блоки, содержащие:

- данные об игроках (имя, цвет фигур),
- отображение последнего совершенного хода.

2.4.3 Система управления ходами

Для предотвращения некорректных действий пользователя предусмотрена система управления ходами, включающая автоматическую блокировку интерфейса во время хода соперника, а также визуальное выделение доступных для перемещения фигур и возможных клеток.

Защита от ошибочных действий реализуется за счет:

- проверки допустимости хода в соответствии с правилами шахмат;
- ограничения действий пользователя только в рамках его очереди;
- обработки особых игровых ситуаций (рокировка, взятие на проходе, превращение пешки, шах).

3 РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА

3.1 Реализация сетевой части

Сетевое взаимодействие в разрабатываемой игре «Шахматы» реализовано с использованием комбинированного подхода, сочетающего преимущества протоколов UDP и TCP. Данное решение обеспечивает эффективный поиск игровых серверов в локальной сети и надёжную передачу игровых команд между участниками.

Функционал клиентского приложения включает:

1 Сканирование локальной сети:

– реализован механизм UDP-широковещательного запроса на порт 8887 с использованием класса `ServerDiscoveryImpl`:

```
socket = new DatagramSocket();  
socket.setBroadcast(true);  
socket.send(new DatagramPacket(queryData, queryData.length,  
    InetAddress.getByName("255.255.255.255"),  
    DISCOVERY_PORT));
```

– сервер отвечает на запрос сообщением в формате JSON, содержащим информацию о каждом активном лобби: имя, адрес, порт, текущее и максимальное количество игроков;

– клиент в течение настраиваемого интервала (по умолчанию 30 секунд) прослушивает входящие ответы и десериализует их в объекты `ServerInfo`;

– полученные данные о доступных серверах публикуются через шину событий `ApplicationEventBus` и отображаются в графическом интерфейсе;

2 Подключение к выбранному серверу:

– для подключения к выбранному серверу создаётся TCP-сокет с настраиваемым таймаутом:

```
Socket socket = new Socket(serverInfo.getAddress(),  
    serverInfo.getPort());  
socket.setSoTimeout(NETWORK_TIMEOUT);
```

– производится отправка команды `ConnectServer`, содержащей имя игрока;

– сервер в ответ направляет `ConnectServerResponse` с подтверждением подключения;

3 Получение и обработка данных:

– создаётся отдельный поток для непрерывного приёма данных от сервера;

– реализован цикл чтения JSON-строк из входного потока сокета с обработкой таймаутов;

– входящие команды десериализуются через фабрику команд `ChessCommandFabric` и маршрутизируются соответствующим клиентским процессорам;

– процессоры публикуют события в `ApplicationEventBus` [6], на которые подписаны UI-контроллеры, обеспечивая обновление графического интерфейса без прямой зависимости от сетевого слоя;

4 Отправка действий игрока:

– игровые действия пользователя упаковываются в объект `ActionCommand`, содержащий начальную и конечную позиции фигуры, а также тип фигуры для превращения пешки;

– команда сериализуется в JSON-конверт класса `Envelope` и отправляется серверу через `PrintWriter`:

```
String responseJson = mapper.writeValueAsString(response);
out.println(responseJson);
```

Серверное приложение обеспечивает:

1 Управление подключениями:

– прослушивание TCP-порта 8888 с использованием `ServerSocket`:

```
ServerSocket serverSocket = new ServerSocket(port);
serverSocket.setSoTimeout(NETWORK_TIMEOUT);
```

– для каждого входящего подключения создаётся экземпляр обработчика `ClientHandlerImpl` с уникальным идентификатором пользователя;

– обработка клиентов выполняется в виртуальных потоках Java 21, что обеспечивает эффективное масштабирование;

– предусмотрено создание лобби командой `CreateLobby` с указанием имени лобби, имени хоста и максимального количества игроков;

– подключение к существующему лобби командой `JoinLobby` с валидацией лимита участников;

2 Координацию игрового процесса:

– создание экземпляра класса `GameService` для обработки игровой логики;

– маршрутизация команд между участниками через словарь серверных процессоров, реализующих интерфейс `ServerCommandProcessor`;

– рассылка обновлений состояния игры всем участникам лобби методом `broadcast`:

```
members.forEach(memberId -> {
    ClientHandler handler = clients.get(memberId);
    if (handler != null) {
        handler.send(command);
    }
});
```

– при выполнении хода сервер рассылает команду `UpdateGameState`, содержащую обновлённый набор фигур на доске и историю ходов;

– при изменении состава участников рассылается команда `UpdateMembers`;

3 Оповещение о доступности сервера:

- класс `DiscoveryResponderImpl` прослушивает UDP-порт 8887 и ожидает запрос обнаружения;
- при получении магической строки `CHESS_DISCOVERY_QUERY` формирует ответ для каждого активного лобби:

```
ServerInfo info = ServerInfo.builder()  
    .name(lobbyData.getName())  
    .hostId(lobbyData.getHost().getId())  
    .port(8888)  
    .currentPlayers(lobbyData.getMembers().size())  
    .maxPlayers(lobbyData.getMaxMemberCount())  
    .build();
```

- ответ сериализуется в JSON и отправляется на адрес запросившего клиента;
- информация о лобби предоставляется независимо от его заполненности.

Для обмена данными между клиентом и сервером используется JSON-формат, включающий:

- тип выполняемого действия (идентификатор команды);
- идентификационные данные игрока (уникальный UUID);
- параметры выполняемой команды (позиции фигур, тип превращения, имя лобби);
- дополнительные служебные данные (идентификатор команды для сопоставления запросов и ответов).

3.2 Реализация игровой логики

Игровая логика шахматного приложения полностью вынесена в общий модуль `shared`, что позволяет использовать единые правила как на серверной стороне при валидации ходов, так и на клиентской при отображении возможных перемещений фигур. Архитектура игровой логики включает следующие компоненты:

1 Модель игрового состояния:

- `GameData` агрегирует все данные партии: словарь `Map<CellPosition, FigureData>` с текущей расстановкой фигур, список истории ходов `List<HistoryData>`, ссылки на белого и чёрного игроков, текущее состояние игры (`GameState`);
- очередность хода определяется по последней записи в истории: если список пуст (ход белых, иначе цвет текущего игрока противоположен цвету последнего хода);
- `CellPosition` моделирует клетку доски (строка и столбец от 1 до 8) и предоставляет геометрические методы `isSameRow`, `isSameColumn`,

isSameDiagonal, isColumnBetween, isRowBetween, используемые при валидации ходов;

- HistoryData хранит запись одного хода: тип действия, цвет и тип фигуры, начальную и конечную позиции, признак рокировки, взятия, шаха или мата, а также метод notation() для формирования шахматной нотации.

2 Иерархия фигур:

- все фигуры наследуются от абстрактного класса FigureData, содержащего общие атрибуты: позицию, цвет (ColorType), флаги moved и dead, тип фигуры (FigureType);

- ключевые методы, переопределяемые в наследниках: validateAction(GameData, ActionCommand), проверка допустимости хода, isUnderAttack(GameData, target, exclude, replaced), определение, атакует ли фигура клетку;

- конкретные реализации для каждого типа: PawnData, RookData, KnightData, BishopData, QueenData, KingData;

- общий алгоритм validateAction включает: проверку выхода за доску, геометрической допустимости хода, отсутствия фигур на пути (кроме коня), отсутствия своей фигуры на целевой клетке, безопасности короля после хода.

3 Специальные правила:

- рокировка (KingData.tryCastling): проверяется, что король и ладья не ходили, ладья жива, между ними нет фигур, король не под шахом и клетки на пути не атакованы; при успехе формируются два действия перемещения;

- взятие на проходе (PawnData.isEnPassant): анализируется последний ход противника, если пешка сходила на две клетки и оказалась рядом с пешкой текущего игрока, разрешается атака по диагонали;

- превращение пешки: при достижении последней горизонтали формируется действие TransformFigure, заменяющее пешку фигурой выбранного типа с сохранением позиции и цвета.

4 Система действий и валидации:

- ActionResult инкапсулирует результат проверки: список ошибок List<ValidationError> и список действий List<ActionPerform>;

- интерфейс ActionPerform с методом doAction(GameData) реализован тремя классами: MoveFigure (перемещение фигуры и установка флага moved), KillFigure (помечает фигуру мёртвой и удаляет с доски), TransformFigure (замена пешки на новую фигуру);

- при валидном ходе действия последовательно применяются к GameData, формируя итоговое изменение состояния.

5 Серверная обработка ходов:

- GameService получает ActionCommand, находит фигуру по начальной позиции и вызывает validateAction;

- при наличии ошибок клиенту возвращается ErrorResponse с описанием проблемы;

- при успехе действия применяются к GameData, формируется запись в HistoryData, и через TCPServer.broadcast() рассылается UpdateGameState всем участникам лобби;
- вся игровая логика выполняется централизованно на сервере, клиенты получают готовое состояние для отображения.

3.3 Реализация пользовательского интерфейса

Интерфейс приложения реализован с использованием фреймворка JavaFX и включает несколько ключевых компонентов:

1 Главное меню (MainScreen).

Содержит три основные кнопки:

- кнопка «Создать игру» открывает диалог настройки новой игровой сессии,
- кнопка «Подключиться к игре» запускает процесс поиска доступных серверов в локальной сети,
- кнопка «Правила игры» отображает справочную информацию.

2 Диалог создания игры:

- позволяет задать имя игрока, название лобби и максимальное количество участников (могут быть не только игроки, но и наблюдатели),
- отправляет серверу команду CreateLobby,
- при успешном создании переводит пользователя на экран лобби в роли хоста.

3 Поиск и подключение к игре:

- запускает UDP-широковещательный запрос для обнаружения серверов,
- отображает индикатор состояния поиска (начало, сканирование, завершение),
- список найденных лобби обновляется динамически по мере поступления ответов,
- для каждого лобби отображается название, количество игроков и кнопка подключения,
- при выборе лобби отправляется команда JoinLobby, и пользователь переходит на экран лобби.

4 Экран лобби:

- отображает список подключённых участников,
- автоматически обновляется при получении UpdateMembersEvent,
- хост видит кнопку запуска игры, активную после набора нужного количества игроков,
- при запуске отправляется команда StartGame, и все участники переходят на игровой экран.

5 Игровая доска:

- представляет собой сетку 8×8 клеток, раскрашенных в шахматном порядке,

- каждая фигура размещается на клетке согласно данным из FigureData,
- доска автоматически перерисовывается при получении UpdateGameStateEvent,

- поддерживает отображение координат (a-h, 1-8) по краям доски.

6 Система отображения фигур:

- каждая фигура представлена объектом Figure, содержащим ImageView с соответствующим изображением,

- фигуры являются интерактивными элементами, реагирующими на клики мыши,

- при выборе своей фигуры подсвечиваются допустимые для хода клетки,

- для фигур противника взаимодействие заблокировано проверкой isMine().

7 Механика взаимодействия:

- выбор фигуры осуществляется кликом мыши по клетке,

- ход совершается кликом по допустимой клетке, после чего формируется ActionCommand,

- команда упаковывается в JSON-конверт Envelope [7] и отправляется серверу,

- при достижении пешкой последней горизонтали появляется диалог выбора фигуры для превращения.

8 Диалог выбора фигуры для превращения пешки:

- появляется автоматически при достижении пешкой последней горизонтали,

- предлагает выбор из четырёх фигур: ферзь, ладья, слон, конь,

- выбранный тип передаётся в поле transform команды ActionCommand.

9 Окно ошибок:

- реализовано с использованием стандартного диалога JavaFX Alert с типом ERROR,

- отображается при получении команды ErrorResponse от сервера,

- информирует о невозможном ходе, заполненном лобби, сетевых сбоях и других проблемах.

4 ТЕСТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

4.1 Юнит-тестирование игровой логики

Целью тестирования является верификация корректности работы игровой логики шахматного приложения, в частности проверка правильности обработки всех типов фигур, контроль валидации игровых ходов, тестирование специальных правил (рокировка, взятие на проходе, превращение пешки).

Методология тестирования построена на модульных тестах с использованием фреймворка JUnit 5 [8]. Базовый класс BaseFigureTest предоставляет общую инфраструктуру для построения тестовых игровых ситуаций: fluent-интерфейс BoardBuilder позволяет в цепочечном стиле расставлять фигуры на доске, задавая их тип, цвет, позицию и признак совершённого хода. Специальные assert-методы (assertError, assertMove, assertKill, assertTransform) проверяют, что результат валидации содержит ожидаемый набор ошибок или действий.

Для каждого типа фигур разработан набор тестовых случаев, разделённый на две группы:

- тесты валидации ходов (ValidateAction): проверка корректного перемещения, выхода за пределы доски, невозможного хода, блокировки другими фигурами, атаки фигуры противника, безопасности короля после хода;

- тесты определения атакованных клеток (isUnderAttack): проверка корректного определения угрозы по горизонтали, вертикали и диагонали, учёт блокировки другими фигурами, проверка при смещении короля.

Для каждой фигуры проверяются специфические правила:

- пешка: ход на одну и две клетки, блокировка после первого хода, атака по диагонали, взятие на проходе, превращение на последней горизонтали;

- ладья: перемещение по вертикали и горизонтали, блокировка фигурами на пути;

- конь: ход буквой «Г», перепрыгивание через другие фигуры;

- слон: перемещение по диагонали, блокировка фигурами на пути;

- ферзь: комбинация ходов ладьи и слона;

- король: перемещение на одну клетку, рокировка (короткая и длинная), проверка, что король и ладья не двигались, ладья жива, клетки не под атакой, отсутствие фигур между ними.

Тестирование валидации ходов включает:

- проверку геометрической допустимости хода для каждого типа фигуры,

- проверку блокировки пути другими фигурами,

- проверку невозможности хода на клетку, занятую своей фигурой,

- проверку, что после хода собственный король не остаётся под шахом,

– тестирование специальных условий (рокировка при атакованном короле, взятие на проходе, превращение пешки).

Пример тест-кейса для хода слона:

```
@Test
@Order(1)
@DisplayName("Bishop movement diagonal left down test")
void moveSuccessDiagonalLeftDownTest() {
    CellPosition testBishopPosition = new CellPosition(5, 5);
    GameData gameData = prepareBoard()
        .with(WHITE, KING).at(1,1)
        .with(WHITE, BISHOP).at(testBishopPosition)
        .build();

    BishopData bishopData = (BishopData)
gameData.getFigureAt(testBishopPosition);
    ActionCommand actionCommand = ActionCommand.builder()
        .fromPosition(testBishopPosition)
        .toPosition(testBishopPosition.shift(-3, -3))
        .build();

    ActionResult result = bishopData.validateAction(gameData,
actionCommand);
    assertMove(bishopData, actionCommand.getToPosition(),
result.getUpdates().getFirst());
}
```

4.2 Интеграционное тестирование

Задачей интеграционного тестирования выступала проверка слаженности работы всех модулей системы в условиях, приближенных к реальной игре.

Проверка проводилась по следующим направлениям.

Подключение участников к игровой сессии:

- хост формирует лобби, задавая имя комнаты и лимит игроков;
- второй участник выполняет поиск доступных серверов и присоединяется к созданной комнате;
- контролировалась корректность начальной инициализации: автоматическая расстановка тридцати двух фигур на стандартные позиции, случайное назначение сторон, передача клиентам информации об именах соперников и цвете фигур.

Проверка ключевых игровых ситуаций:

Перемещение фигуры: отслеживалось, что после совершения хода одним игроком доска второго обновляется в идентичное состояние, право хода переходит корректно.

Захват вражеской фигуры: проверялось исчезновение взятой фигуры с поля у обеих сторон.

Ситуация шаха и мата: верифицировалось определение состояния шаха, невозможность совершить ход, открывающий собственного короля, и фиксация завершения партии при матовой позиции [9].

Выполнение рокировки: контролировалась одновременная смена позиции короля и ладьи на экранах обоих участников, а также отклонение рокировки системой при нахождении клеток под ударом.

Замена пешки при достижении края доски: проверялся вызов окна выбора новой фигуры, применение выбранного варианта и идентичность итогового состояния у соперников.

Взятие на проходе: тестировалась обработка данного правила, а именно удаление пешки противника с корректной клетки и совпадение отображения у обеих сторон.

Среда выполнения тестов:

- два компьютера, соединённых локальной сетью;
- обмен данными по протоколу TCP через порт 8888;
- обнаружение игрового сервера широковебательными UDP-запросами на порт 8887.

Полученные результаты:

Установление связи:

- TCP-подключение формировалось менее чем за секунду;
- поиск сервера через UDP завершался в течение одной-двух секунд;
- начальная расстановка синхронизировалась практически мгновенно.

Игровое взаимодействие:

- действия соперника отображались с задержкой порядка 100-200 миллисекунд;
- нарушений очередности ходов не выявлено;
- расхождений в состоянии доски между клиентами не зафиксировано.

5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

5.1 Описание пользовательского интерфейса

Главный экран приложения встречает пользователя фоновым изображением и двумя крупными кнопками: «Создать игру» и «Присоединиться». Кнопка «Создать игру» открывает диалог для настройки новой игровой сессии, где пользователь становится хостом. Кнопка «Присоединиться» ведёт на экран поиска активных серверов в локальной сети. Главный экран изображен на рисунке 3.

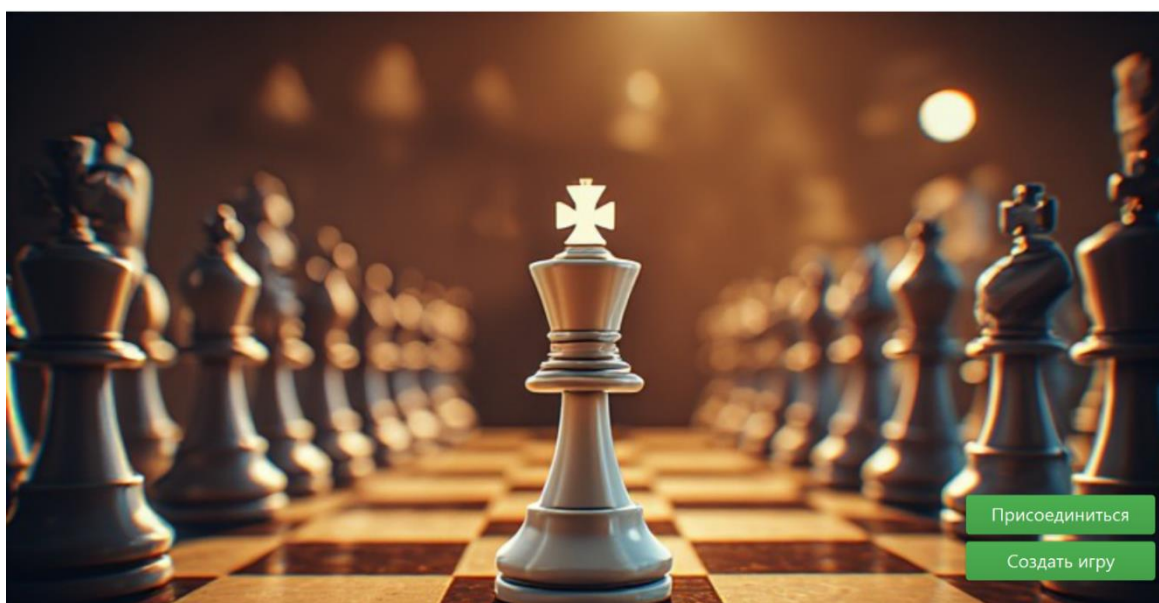


Рисунок 3 – Главный экран приложения

При создании игры открывается диалог, где пользователь вводит название комнаты, своё имя и выбирает максимальное количество участников. Подтверждение переносит его в комнату ожидания в роли хоста. Диалог создания комнаты изображен на рисунке 4, комната ожидания представлена на рисунке 5.

Скриншот диалога создания комнаты. Вверху — стандартные кнопки управления окном (минус, квадрат, крест). В центре — три поля ввода: «Лобби:» с текстом «Лобби», «Имя:» с текстом «Катя», «Число игроков:» со значением «2» и стрелками для изменения. Внизу — две зеленые кнопки: «Отмена» и «Создать».

Рисунок 4 – Диалог создания комнаты

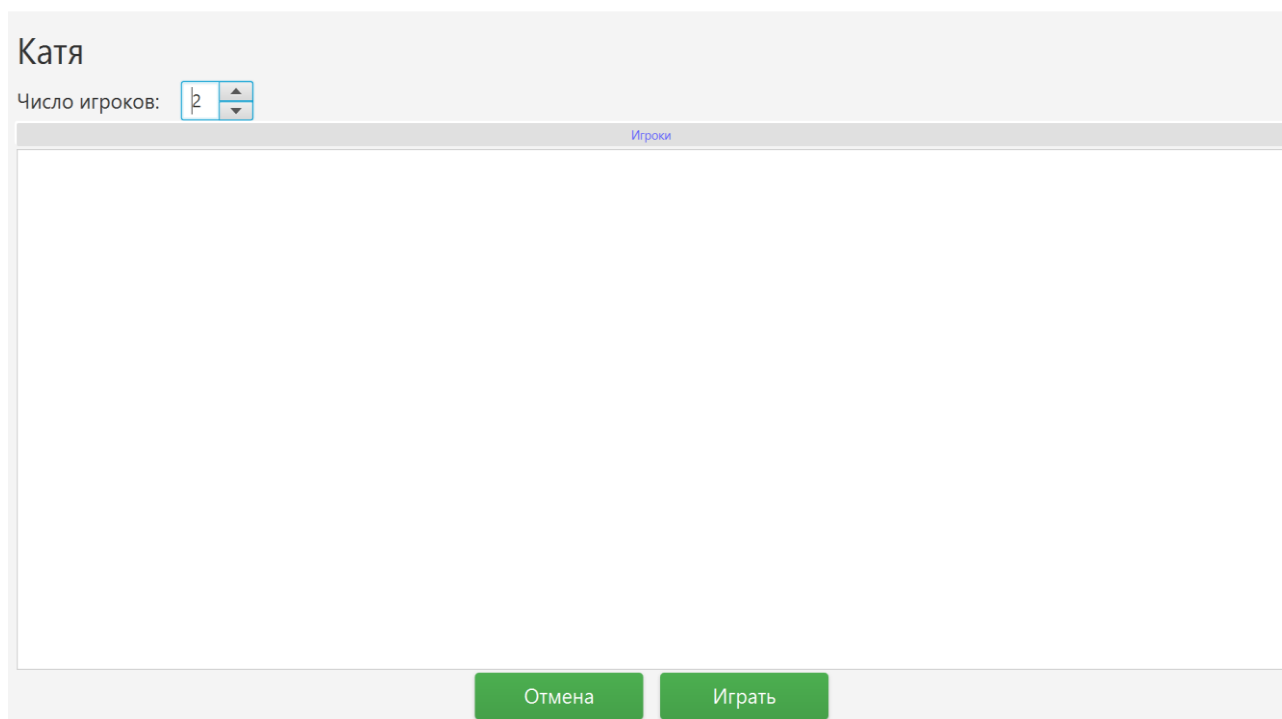


Рисунок 5 – Комната ожидания

В комнате ожидания показывается имя создателя комнаты и список всех подключённых участников. Создатель комнаты (хост) может изменить максимальное количество игроков, нажать кнопку «Играть», которая становится активной при наборе нужного числа участников. Нажатие на неё запускает партию: сервер случайно распределяет цвета, расставляет фигуры в начальную позицию и переводит всех на игровой экран.

При нажатии кнопки «Присоединиться» на главном экране открывается экран поиска серверов, где в таблице отображаются найденные в локальной сети лобби с указанием названия, IP-адреса и количества игроков.

Список заполняется динамически по мере получения ответов от серверов. Выбрав строку таблицы и нажав «Войти», пользователь указывает своё имя в дополнительном диалоге, после чего присоединяется к выбранной комнате. Экран поиска серверов изображен на рисунке 6, дополнительный диалог представлен на рисунке 7.

Имя	IP	Онлайн
Лобби	10.35.14...	0 / 2
Лобби	127.0.0.1	0 / 2

Рисунок 6 – Экран поиска серверов

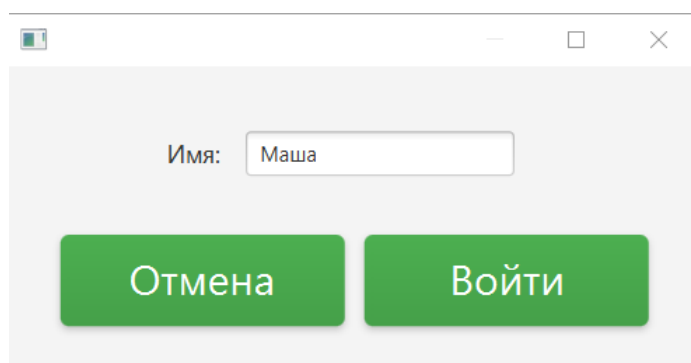


Рисунок 7 – Дополнительный диалог

Вход игрока в комнату, созданную ранее другим игроком, изображен на рисунке 8.

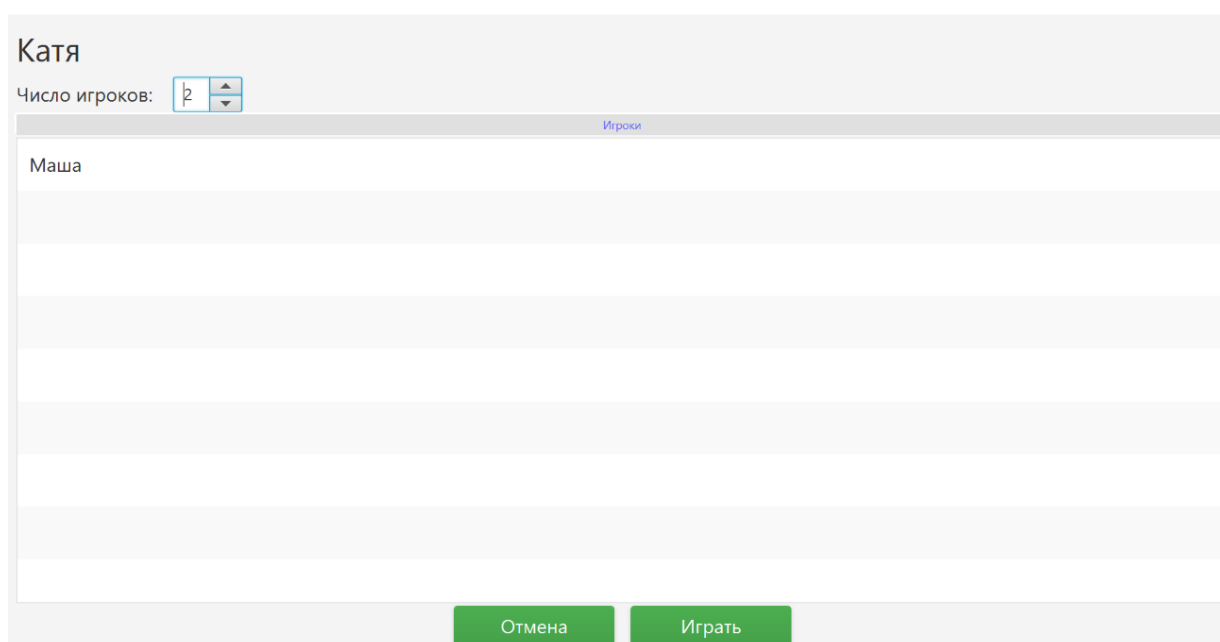


Рисунок 8 – Вход игрока в комнату

Центральное место на игровом экране занимает шахматная доска с расставленными фигурами. Имя игрока, играющего чёрными, отображается над доской, имя игрока играющего белыми под доской.

Игрок кликом выбирает свою фигуру, видит подсвеченные допустимые ходы и кликает по целевой клетке. Выбор чужих фигур во время хода соперника заблокирован.

Справа от доски расположена панель истории, в которую автоматически добавляются записи о каждом совершённом ходе в шахматной нотации [10] (шах обозначается знаком «+», мат – знаком «#», взятие фигуры – «x» и т.д.). Кнопка «Выйти» завершает игру и возвращает в главное меню. Игровой экран представлен на рисунке 9.

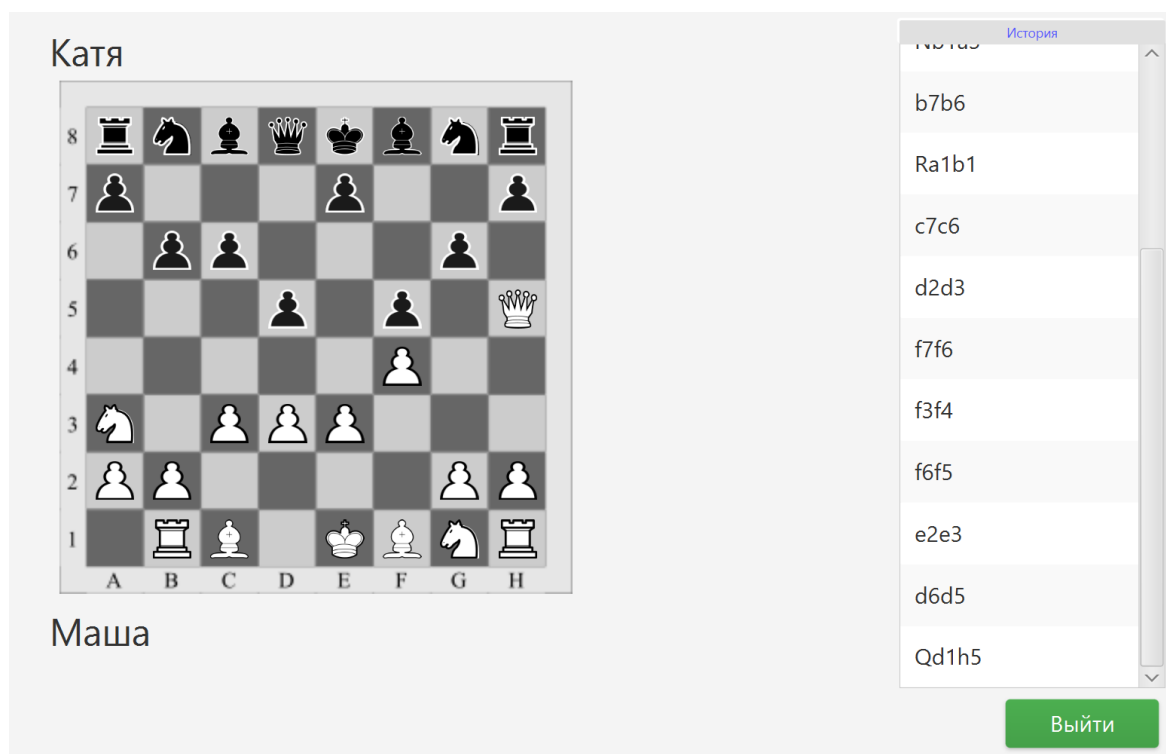


Рисунок 9 – Игровой экран

При достижении пешкой последней горизонтали поверх игрового экрана открывается диалог выбора новой фигуры с четырьмя вариантами: ладья, конь, слон и ферзь [11]. Каждый представлен кнопкой с соответствующим изображением. После выбора пешка заменяется указанной фигурой, и игра продолжается.

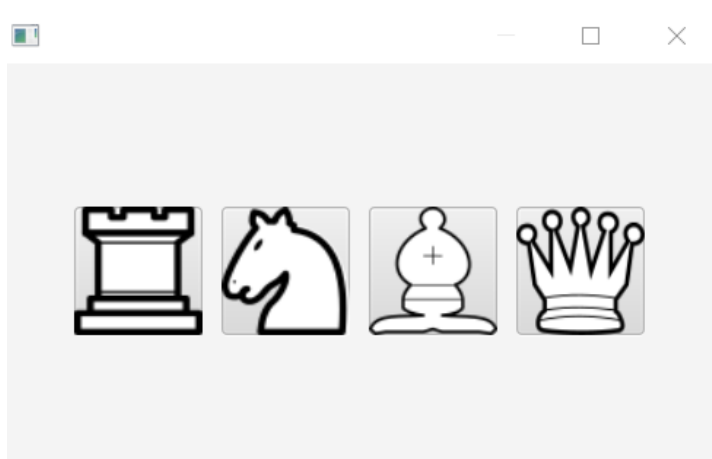


Рисунок 10 – Диалог выбора новой фигуры

5.2 Управление программным средством

Управление приложением осуществляется через графический интерфейс с помощью мыши и клавиатуры [12]. Взаимодействие пользователя с системой включает следующие основные сценарии:

1 Запуск серверной части.

Сервер запускается как отдельное приложение без графического интерфейса. При старте он инициализирует внутренние компоненты и открывает два сетевых порта: для приёма подключений игроков и для ответа на поисковые запросы в локальной сети. После этого сервер готов к созданию комнат и ожидает подключений.

2 Подключение клиента.

Клиентская часть запускается как приложение с графическим интерфейсом. Пользователь попадает на главный экран, где может создать новую игру или присоединиться к существующей. При создании игры открывается диалог с полями для ввода названия комнаты, имени и выбора количества участников. При подключении к игре запускается поиск серверов, результаты отображаются в таблице с названием, адресом и количеством игроков. Выбрав комнату и указав имя, пользователь присоединяется к ней.

3 Управление игровыми комнатами.

Управление комнатой доступно только владельцу комнаты (хосту). На экране лобби он видит список участников, который обновляется автоматически, и может изменить максимальное количество игроков. Когда набирается нужное число участников, хост нажимает кнопку «Играть», сервер распределяет цвета, расставляет фигуры и запускает партию.

4 Управление игровым процессом.

Игровой процесс сводится к выбору фигуры и указанию целевой клетки. Первым кликом игрок выбирает свою фигуру, вторым кликом по клетке ход отправляется серверу. Если на целевой клетке стоит фигура противника, она забирается. При достижении пешкой края доски появляется окно выбора новой фигуры с четырьмя вариантами. Во время хода соперника фигуры текущего игрока заблокированы.

5 Проверка корректности действий.

Все действия игрока проверяются сервером. Нельзя нарушить правила перемещения фигур, походить вне очереди или оставить своего короля под шахом. Запрещено подключиться к заполненной комнате или запустить игру, не являясь хостом. При любом некорректном действии появляется окно с описанием ошибки.

6 Завершение игры.

Игра завершается матом, ничьей или добровольным выходом игрока. Игрок может покинуть партию в любой момент кнопкой «Выйти». После завершения пользователь возвращается в главное меню.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была успешно разработана и реализована сетевая компьютерная версия игры в шахматы, полностью соответствующая поставленным техническим требованиям и официальным правилам игры.

Основные достижения включают: создание полнофункционального игрового цикла с поддержкой всех типов фигур (пешка, ладья, конь, слон, ферзь, король) и реализацией специальных правил (рокировка, взятие на проходе, превращение пешки), разработку системы валидации ходов с проверкой шаха и мата, а также создание стабильного сетевого взаимодействия для двух игроков с использованием комбинации TCP и UDP протоколов, включая механизмы обнаружения серверов в локальной сети, синхронизацию игрового состояния и корректную обработку разрывов соединения.

Пользовательский интерфейс был спроектирован с учётом принципов удобства и наглядности: главный экран с возможностью создания игры или подключения к существующей, экран лобби с отображением участников и кнопкой запуска для хоста, игровой экран с шахматной доской и панелью истории ходов в стандартной нотации, диалог выбора фигуры при превращении пешки, а также окна ошибок с понятными сообщениями. Проведённое тестирование (модульные тесты игровой логики для каждого типа фигур, интеграционные проверки сетевого взаимодействия) подтвердило корректность работы всех модулей, стабильность системы и отсутствие расхождений в состоянии доски между клиентами.

Разработанное программное обеспечение демонстрирует полное соответствие техническому заданию, надёжность работы в локальной сети и удобство взаимодействия для пользователей.

Перспективы развития проекта включают добавление возможности игры с компьютерным соперником на основе шахматных алгоритмов, доработку интерфейса для запуска на мобильных устройствах, что позволит расширить функциональность приложения и привлечь дополнительную аудиторию.

Проведённая работа подтвердила возможность создания качественной компьютерной реализации классической настольной игры с сохранением всех ключевых особенностей оригинальных шахмат, а также продемонстрировала эффективность выбранных технологий (Java, JavaFX) для разработки сетевых приложений данного класса.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Приложение Chess – Play and Learn [Электронный ресурс]. – Режим доступа: <https://www.chess.com/learn>.
- [2] Приложение Real Chess [Электронный ресурс]. – Режим доступа: https://play.google.com/store/apps/details?id=com.Alienforcegames.ChessFusionFree&hl=en_CA.
- [3] Классы Socket и ServerSocket в Java [Электронный ресурс]. – Режим доступа: <https://javarush.com/groups/posts/654-klassih-socket-i-serversocket-ili-allo-server-tih-menja-slihshishjh>.
- [4] Data Transfer Object DTO Definition and Usage [Электронный ресурс]. – Режим доступа: <https://www.okta.com/identity-101/dto/>.
- [5] JavaFX Documentation [Электронный ресурс]. – Режим доступа: <https://openjfx.io/#>.
- [6] How it works? EventBus [Электронный ресурс]. – Режим доступа: <https://medium.com/@sadeqh.dehghani1992/how-it-works-eventbus-cef03ac2a12f>.
- [7] Introduction to JSON-Java [Электронный ресурс]. – Режим доступа: <https://www.baeldung.com/java-org-json>.
- [8] JUnit 5 User Guide [Электронный ресурс]. – Режим доступа: <https://docs.junit.org/5.5.0/user-guide/>.
- [9] Матовые сети: шахматная тактика [Электронный ресурс]. – Режим доступа: <https://chessrussian.ru/materialy/beginners/matovaya-set-zazhatyy-korol/>.
- [10] Нотация в шахматах: основы тактика [Электронный ресурс]. – Режим доступа: <https://chessrussian.ru/materialy/beginners/notatsiya-dlya-nachinayushchikh-igrokov-v-shakhmaty/>.
- [11] Превращение пешки тактика [Электронный ресурс]. – Режим доступа: <https://www.chess.com/ru/terms/prevrashchenie-peshki>.
- [12] JavaFX guide: Go graphical with Java [Электронный ресурс]. – Режим доступа: <https://bell-sw.com/blog/javafx-guide-go-graphical-with-java/>.

ПРИЛОЖЕНИЕ А. Исходный код программы

```
server launcher
package com.nargiz.chess.server;

import com.nargiz.chess.server.network.TCPServer;
import com.nargiz.chess.shared.ioc.Container;
import com.nargiz.chess.shared.ioc.Context;

public class ServerLauncher {

    public static void main(String[] args) {
        // создаём ioc контейнер и сканируем компоненты
        Context context = Container.createContext("com.nargiz.chess");

        // получаем tcp сервер из контейнера
        TCPServer server = context.get(TCPServer.class);

        System.out.println(server);

        // старт сервера на порту 8888
        server.start(8888);
    }
}

shared/constants/networkconstants.java
package com.nargiz.chess.shared.constants;

public interface NetworkConstants {

    // порт для udp обнаружения серверов
    int DISCOVERY_PORT = 8887;

    // строка запроса для поиска серверов в сети
    String DISCOVERY_QUERY = "CHESS_DISCOVERY_QUERY";

    // сколько секунд сервер отвечает на discovery
    int discoveryDurationSeconds = 30;
}

shared/models/serverinfo.java
package com.nargiz.chess.shared.models;

import lombok.*;

import java.util.UUID;

@Getter
```

```

@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
@ToString
public class ServerInfo {

    // имя сервера (лобби)
    private String name;

    // ip адрес сервера
    private String address;

    // id хоста
    private UUID hostId;

    // tcp порт
    private int port;

    // текущее количество игроков
    private int currentPlayers;

    // максимальное число игроков
    private int maxPlayers;
}
tcpserver.java
package com.nargiz.chess.server.network;

public interface TCPServer {

    // запуск tcp сервера
    void start(int port);
}
tcpserverimpl.java

package com.nargiz.chess.server.network.impl;

import com.nargiz.chess.server.network.TCPServer;

import com.nargiz.chess.shared.ioc.anoation.Component;

@Component
public class TCPServerImpl implements TCPServer {

    @Override

    public void start(int port) {

```

```

        System.out.println("tcp server started on port: " + port);
    }
}

discovery responder

package com.nargiz.chess.server.network;

public interface DiscoveryResponder {

    // отвечает на udp запросы клиентов

    void start();

}

shared/command/envelope.java

package com.nargiz.chess.shared.command;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.NoArgsConstructor;
import java.util.HashMap;
import java.util.Map;

@Data
@NoArgsConstructor
@AllArgsConstructor
@NoArgsConstructor

public class Envelope {

    // имя команды (createLobby, action, join и тд)

    String command;

    // payload команды (динамический json)

    Map<String, Object> body = new HashMap<>();

}

chesscommand.java

package com.nargiz.chess.shared.command;

```

```

import lombok.*;

import lombok.experimental.SuperBuilder;

import java.util.UUID;

@Setter

@Getter

@ToString

@SuperBuilder

@NoArgsConstructor

@AllArgsConstructor

public class ChessCommand {

    // уникальный id команды (для трекинга)

    @Builder.Default

    UUID commandId = UUID.randomUUID();

    // кто отправил команду

    UUID userId;

}

actioncommand.java

package com.nargiz.chess.shared.command;

import com.nargiz.chess.shared.models.CellPosition;

import com.nargiz.chess.shared.models.enums.FigureType;

import lombok.*;

@Getter

@SuperBuilder

@NoArgsConstructor

@AllArgsConstructor

@ToString(callSuper = true)

public class ActionCommand extends ChessCommand {

    // превращение пешки

```

```

        FigureType transform;

        // откуда ход

        CellPosition fromPosition;

        // куда ход

        CellPosition toPosition;
    }

connectserver.java

package com.nargiz.chess.shared.command;

import lombok.*;

@Getter

@SuperBuilder

@NoArgsConstructor

@AllArgsConstructor

@ToString(callSuper = true)

public class ConnectServer extends ChessCommand {

    // имя игрока при подключении

    String memberName;
}

```

```

createlobby.java

package com.nargiz.chess.shared.command;

import lombok.*;

@Getter

@SuperBuilder

@NoArgsConstructor

@AllArgsConstructor

@ToString(callSuper = true)

public class CreateLobby extends ChessCommand {

```

```

    String memberName;

    String lobbyName;

    int maxPlayerCount;
}

joinlobby.java

package com.nargiz.chess.shared.command;

import lombok.*;

import java.util.UUID;

@Getter

@SuperBuilder

@NoArgsConstructor

@AllArgsConstructor

@ToString(callSuper = true)

public class JoinLobby extends ChessCommand {

    String memberName;

    // к какому хосту подключаемся

    UUID hostId;

}

startgame.java

package com.nargiz.chess.shared.command;

import lombok.*;

import java.util.UUID;

@Getter

@SuperBuilder

@NoArgsConstructor

@AllArgsConstructor

@ToString(callSuper = true)

public class StartGame extends ChessCommand {

```

```

    // кто запускает игру
    UUID memberId;
}

chesscommandresponse.java

package com.nargiz.chess.shared.command.response;

import com.nargiz.chess.shared.command.ChessCommand;
import lombok.*;

@SuperBuilder
@NoArgsConstructor

public class ChessCommandResponse extends ChessCommand {

    // базовый ответ сервера
}

connectserverresponse.java

@Data
@SuperBuilder
@NoArgsConstructor
@AllArgsConstructor

public class ConnectServerResponse extends ChessCommandResponse {

    // разрешён ли вход
    private boolean allowed;

    // сообщение
    private String message;
}

createlobbyresponse.java

@Data
@SuperBuilder
@NoArgsConstructor

```


@AllArgsConstructor

```
public class CreateLobbyResponse extends ChessCommandResponse {
```

```
    private UUID userId;
```

```
    private String hostName;
```

```
    private int maxMemberCount;
```

```
    private String message;
```

```
}
```

joinlobbyresponse.java

@Data

@SuperBuilder

@NoArgsConstructor

@AllArgsConstructor

```
public class JoinLobbyResponse extends ChessCommandResponse {
```

```
    private UUID userId;
```

```
    private String hostName;
```

```
    private String message;
```

```
}
```

startgamerresponse.java

@Data

@SuperBuilder

@NoArgsConstructor

@AllArgsConstructor

```
public class StartGameResponse extends ChessCommandResponse {
```

```
    private UUID whiteUserId;
```

```
    private UUID blackUserId;
```

```
    private String whitePlayerName;
```

```
    private String blackPlayerName;
```

```
    // стартовая расстановка фигур
```

```

        private Collection<FigureData> figures;
    }

errorresponse.java

@Data
@SuperBuilder
@NoArgsConstructor
@AllArgsConstructor

public class ErrorResponse extends ChessCommandResponse {

    // текст ошибки

    private String message;
}

Figuredto

package com.nargiz.chess.shared.models.figures;

import com.nargiz.chess.shared.models.CellPosition;

import com.nargiz.chess.shared.models.enums.ColorType;

import com.nargiz.chess.shared.models.enums.FigureType;

import lombok.*;

import java.util.UUID;

@Getter

@Setter

@NoArgsConstructor

@AllArgsConstructor

@ToString

public class FigureDTO {

    // уникальный id фигуры

    private UUID id;

    // позиция на доске

    private CellPosition position;

```

```

    // цвет фигуры
    private ColorType color;

    // тип фигуры (king, queen...)
    private FigureType figureType;

    // жива ли фигура
    private boolean dead;

    // делала ли ход
    private boolean moved;
}

Figureresolver

package com.nargiz.chess.shared.models.figures;

import com.nargiz.chess.shared.models.FigureData;

public class FigureResolver {

    // перевод доменной фигуры в сетевую модель
    public static FigureDTO toDTO(FigureData figure) {

        if (figure == null) return null;

        return new FigureDTO(

            figure.getId(),

            figure.getPosition(),

            figure.getColor(),

            figure.getFigureType(),

            figure.isDead(),

            figure.isMoved()

        );

    }

}

package com.nargiz.chess.shared.models.figures;

```

```

public enum FigureSpriteType {

    KING,

    QUEEN,

    ROOK,

    BISHOP,

    KNIGHT,

    PAWN

}

package com.nargiz.chess.shared.models;

import com.nargiz.chess.shared.models.figures.FigureDTO;

import lombok.*;

import java.util.Collection;

@Getter

@Setter

@NoArgsConstructor

@AllArgsConstructor

@ToString

public class GameSnapshot {

    // все фигуры на доске (для клиента)

    private Collection<FigureDTO> figures;

    // чей сейчас ход

    private String currentPlayer;

    // состояние игры

    private String state;

}

Updategamesnapshotresponse

package com.nargiz.chess.shared.command.response;

```

```

import com.nargiz.chess.shared.command.ChessCommand;

import com.nargiz.chess.shared.models.GameSnapshot;

import lombok.*;

@Data
@SuperBuilder
@NoArgsConstructor
@AllArgsConstructor

public class UpdateGameStateResponse extends ChessCommand {

    // полный снимок игры после хода

    private GameSnapshot snapshot;

}

```

ContextTest.java

```

package com.nargiz.chess.shared.ioc;

import com.nargiz.chess.shared.ioc.anoatation.Component;
import com.nargiz.chess.shared.ioc.anoatation.Inject;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class ContextTest {

    @Component

    static class AService {

        public String hello() {

            return "hello";

        }

    }

    @Component

    static class BService {

        @Inject

        private AService aService;

    }

}

```

```

    public String call() {

        return aService.hello();

    }

}

@Test

void shouldInjectDependencies() {

    Context context = Container.createContext("com.nargiz.chess");

    BService b = context.get(BService.class);

    assertNotNull(b);

    assertEquals("hello", b.call());

}

}

```

BishopTest.java

```

package com.nargiz.chess.shared.models.figures;

import com.nargiz.chess.shared.command.ActionCommand;

import com.nargiz.chess.shared.models.*;

import com.nargiz.chess.shared.models.enums.ColorType;

import org.junit.jupiter.api.Test;

import java.util.Map;

import static org.junit.jupiter.api.Assertions.*;

public class BishopTest {

    @Test

    void bishop_should_move_diagonally() {

        BishopData bishop = new BishopData();

        bishop.setPosition(new CellPosition(3, 3));

        bishop.setColor(ColorType.WHITE);

        GameData game = GameData.builder()

            .figures(Map.of(bishop.getPosition(), bishop))

```

```

        .build();

    ActionCommand cmd = ActionCommand.builder()

        .fromPosition(new CellPosition(3, 3))

        .toPosition(new CellPosition(5, 5))

        .build();

    ActionResult result = bishop.validateAction(game, cmd);

    assertTrue(result.getErrors().isEmpty());

    assertFalse(result.getUpdates().isEmpty());

}

@Test

void bishop_should_fail_invalid_move() {

    BishopData bishop = new BishopData();

    bishop.setPosition(new CellPosition(3, 3));

    bishop.setColor(ColorType.WHITE);

    GameData game = GameData.builder()

        .figures(Map.of(bishop.getPosition(), bishop))

        .build();

    ActionCommand cmd = ActionCommand.builder()

        .toPosition(new CellPosition(3, 5))

        .build();

    ActionResult result = bishop.validateAction(game, cmd);

    assertFalse(result.getErrors().isEmpty());

}

}

```

ChessCommandFabricTest.java

```

package com.nargiz.chess.shared.fabric.impl;

import com.nargiz.chess.shared.fabric.ChessCommandFabric;

import com.nargiz.chess.shared.ioc.Container;

```

```

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

public class ChessCommandFabricTest {

    @Test

    void should_find_command_by_name() {

        Container.createContext("com.nargiz.chess");

        ChessCommandFabric fabric =

            Container.getInstance().get(ChessCommandFabric.class);

        Class<?> clazz = fabric.getCommand("ActionCommand");

        assertNotNull(clazz);

        assertEquals("ActionCommand", clazz.getSimpleName());

    }

}

EventBusTest.java

package com.nargiz.chess.shared.events.impl;

import com.nargiz.chess.shared.events.GameEvent;

import org.junit.jupiter.api.Test;

import java.util.concurrent.atomic.AtomicBoolean;

import static org.junit.jupiter.api.Assertions.*;

public class EventBusTest {

    static class TestEvent implements GameEvent {}

    @Test

    void should_publish_event() {

        ApplicationEventBusImpl bus = new ApplicationEventBusImpl();

        AtomicBoolean called = new AtomicBoolean(false);

        bus.subscribeOn(TestEvent.class, e -> called.set(true));

        bus.publish(new TestEvent());

        assertTrue(called.get());
    }
}

```



```

    }
}

GameSnapshotTest.java

package com.nargiz.chess.shared.models;

import com.nargiz.chess.shared.models.figures.FigureDTO;
import org.junit.jupiter.api.Test;
import java.util.List;
import static org.junit.jupiter.api.Assertions.*;

public class GameSnapshotTest {

    @Test

    void snapshot_should_store_figures() {

        GameSnapshot snapshot = new GameSnapshot();

        FigureDTO dto = new FigureDTO();

        dto.setDead(false);

        snapshot.setFigures(List.of(dto));

        assertEquals(1, snapshot.getFigures().size());

    }

}

```